Initial Voiced Consonant Detection and Spectral Analysis in MSWC English

Haotian Fang Ziheng Yuan

April 29, 2026

Abstract

This report studies the VE216 Project Topic 2 speech analysis task and focuses on the initial voiced consonants /g/, /b/, /d/, and /z/. The data come from MSWC English. We keep only words whose first letter is `g/b/d/z` and whose first non-vowel phone is the matching `G/B/D/Z`. Because the /z/ class contains only 75 eligible words, the final dataset uses 75 words per class, for a total of 300 words and 6796 utterances. We first present a detection method based on short-time FFT magnitude spectra and normalized correlation. We then present an AI reimplementation that uses log-mel features, onset localization, positive-negative templates, and autocorrelation. Finally, we compare both template methods with an MLP and a CNN. On the test set, exact-match accuracy is 0.265 for our method, 0.503 for the AI reimplementation, 0.485 for the MLP, and 0.559 for the CNN.

1 Problem Definition, Experimental Setting, and Metrics

Let $x_i[n]$ denote the ith waveform. All recordings are resampled to 16 kHz and trimmed or padded to 1 s. The label set is

$$C = \{g, b, d, z\}, \qquad y_i \in \{0,1\}^{|C|}.$$

The task is single-label four-class classification, and each sample has exactly one initial voiced consonant label. To keep one shared evaluation pipeline for the template methods, the MLP, and the CNN, each method first outputs a score vector $s_i \in \mathbb{R}^{|C|}$ and then decodes it into a predicted label $\hat{y}_i$. Since every sample satisfies $\sum_c y_{i,c} = 1$, the reported evaluation uses exclusive decoding:

$$\hat{c}_i = \arg\max_{c \in C} s_{i,c}, \qquad \hat{y}_{i,c} = \mathbf{1}[c = \hat{c}_i].$$

Per-class thresholds are still estimated on the development set because the code keeps a multi-output interface, but all test results in the report use the single-label decoding above.

All methods use the same train/development/test split. The training set is used only to estimate templates or train model parameters. The development set is used only to choose hyperparameters, thresholds, and early stopping. The test set is used only for the final metrics. No manual phone segmentation is provided, so every method has to locate useful evidence near the beginning of the word from the whole-word recording.

1.1 Evaluation metrics

Let N be the number of test samples and let $K = |C|$. Label-wise accuracy compares each class bit separately:

$$\text{LabelAcc} = \frac{1}{NK} \sum_{i=1}^{N} \sum_{c \in C} \mathbf{1}[\hat{y}_{i,c} = y_{i,c}].$$

This metric is equal to 1 minus Hamming loss. Because it rewards both true positives and true negatives, its value is higher than exact-match accuracy in a single-label task.

Exact-match accuracy requires the whole label vector to be correct:

$$\text{ExactMatch} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\hat{y}_i = y_i].$$

Under the one-hot label setting used here, this metric is the standard four-class accuracy and serves as the main metric in the report.

For each class c, precision, recall, and F1 are defined as

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}.$$

Macro-F1 is the average of the four class-level F1 scores:

$$\text{MacroF1} = \frac{1}{K}\sum_{c\in C} F1_c.$$

It does not weight classes by sample count, so it shows clearly whether one of /g/, /b/, /d/, and /z/ is sacrificed.

Micro-F1 first pools all TP, FP, and FN counts and then computes F1:

$$\text{MicroF1} = \frac{2\sum_c TP_c}{2\sum_c TP_c + \sum_c FP_c + \sum_c FN_c}.$$

Samples-F1 computes F1 for each sample and then averages over samples:

$$\text{SamplesF1} = \frac{1}{N}\sum_{i=1}^{N} \frac{2|Y_i \cap \hat{Y}_i|}{|Y_i| + |\hat{Y}_i|}.$$

Because both the ground-truth label and the predicted label contain exactly one active class, micro-F1, samples-F1, and exact-match accuracy have the same numerical value here. These metrics remain in the tables because the evaluation code uses one shared interface.

The report compares three groups of methods in sequence. It first introduces our method, which uses short-time FFT templates. It then presents the AI reimplementation, which uses a stronger spectral representation and a more local detection strategy. The MLP and CNN appear last as neural baselines so that the main line of the report remains focused on the acoustic evidence itself.

2 Related Work and Theoretical Background

The task studied here is not large-vocabulary recognition. It asks whether a short spoken word begins with one target consonant and, if so, which consonant it is. This setting is closer to short-time analysis, local spectral comparison, and template matching in classical speech processing. The data also come from MSWC, which is a word-level speech corpus designed for keyword spotting and spoken term search[4].

2.1 Short-time analysis and spectral representation

Davis and Mermelstein show that useful speech front ends do not rely on the raw waveform alone. They rely on short-time parameterized representations obtained after framing and spectral analysis[1]. The procedure is simple: the waveform is split into frames of a few tens of milliseconds, each frame is windowed, and a Fourier transform is applied. The resulting spectrum preserves local cues such as stop release, frication, and vowel transition, while reducing the instability of the raw waveform.

Rabiner's tutorial on hidden Markov models connects these frame-level features with temporal modeling[2]. The important point here is that speech is not a static image. Useful evidence lies in how local structure changes over time. Even though this report does not use HMMs, the same idea still matters. If the goal is to detect an initial consonant inside a whole-word recording, it is more suitable to search a short region near the beginning of the word than to average the whole recording into one global descriptor.

2.2 Why consonant detection depends on local spectral evidence

For /g/, /b/, /d/, and /z/, the most useful evidence often appears in a short initial segment. The release of a stop consonant is brief and changes fast, whereas a fricative tends to keep high-frequency energy for a longer time. Once these cues are averaged over the whole word, much of the information disappears. Wilkinson and Russell report a similar pattern in phone recognition on TIMIT: local evidence near acoustic boundaries carries strong value for consonant discrimination[3].

For this reason, the main method in this report does not begin with a complex model. It begins with the most direct signal-processing tools: short-time FFT, local spectra, and normalized correlation. The purpose is not to claim that a simpler method is stronger. The purpose is to make the structure of the task visible before stronger models are introduced.

2.3 Why the report begins with our method

Keyword spotting studies often use small DNN or CNN models[5, 6], finite-vocabulary benchmarks such as Speech Commands[7], transformer models and self-supervised speech representations[8, 9, 10, 11, 12], and streaming open-vocabulary detection[13]. This report still introduces our method first, then presents the AI reimplementation, and finally adds the MLP and CNN baselines. This order makes it easier to explain the spectral cues near the word onset before comparing the gains brought by stronger models.

3 Dataset Construction and Labeling Rules

3.1 Selection rule

The subset used in this report follows one simple rule:

> A word is kept if and only if its first letter is one of `g/b/d/z` and its first non-vowel phone in CMUdict or ARPABET is the matching consonant `G/B/D/Z`.

This rule turns a spelling-based filter into a pronunciation-based filter. It removes words in which the target letter appears in the spelling but does not define the initial consonant. It also separates the location of the written letter from the true acoustic boundary of the initial sound.

A check of the original candidate pool shows that 100 words per class are not possible under this strict rule. If each word only needs at least one utterance in train, development, and test, the numbers of candidate words are 957 for /g/, 2073 for /b/, 1932 for /d/, and only 75 for /z/. The final dataset therefore uses 75 words per class, which is the largest balanced choice allowed by the /z/ class. During sampling, each word keeps at least one utterance in train, development, and test, and the number of utterances per word is capped at 40/5/5 so that the dataset does not concentrate too many samples on a small set of words. The four classes are then truncated to the same sample counts in train, development, and test.

3.2 Dataset size

Table 1: Size statistics of the filtered initial voiced-consonant subset.

Class	Words	Train	Development	Test
/g/	75	1311	194	194
/b/	75	1311	194	194
/d/	75	1311	194	194
/z/	75	1311	194	194
Total	300	5244	776	776

The resulting subset is balanced both in the number of words per class and in the number of utterances per split. The test set contains 194 utterances for each class and 776 utterances in total. This design increases lexical coverage and makes the task closer to cross-word detection of initial voiced consonants rather than recognition of a few representative words.

4 Theory and Two Template Methods

4.1 Our method: short-time FFT templates

Our method corresponds to `strict_course_fft` in the code. Its modeling idea is direct: the useful evidence for the initial voiced consonant lies in a short segment near the word onset, and different classes have different local magnitude-spectrum prototypes. Recording level, speaker identity, and the following vowel all introduce variation, so the method compares normalized local spectral shape rather than whole-word energy.

Let $x_i[n]$ be the waveform of sample i. The frame length is $L = 20$ ms, the hop size is $H = 10$ ms, and the FFT size is $N = 512$. The spectrum of frame m is

$$X_{i,m}[k] = \sum_{n=0}^{L-1} x_i[n + mH]w[n]e^{-j2\pi kn/N}, \qquad k = 0, \ldots, N-1,$$

where $w[n]$ is a Hamming window. Only the one-sided magnitude spectrum is kept, and each frame is normalized by its maximum magnitude. If a candidate block starts at frame r and spans M frames, its descriptor is

$$P_i^{(r)}[k] = \frac{1}{M} \sum_{m=0}^{M-1} \frac{|X_{i,r+m}[k]|}{\max_\kappa |X_{i,r+m}[\kappa]|}, \qquad k = 0, \ldots, N/2.$$

The descriptor $P_i^{(r)}$ is then normalized by its L_2 norm. This step suppresses amplitude variation and keeps the shape of the short-time spectrum.

Templates are estimated from the training data. For each class c, the method first selects the maximum-energy M-frame block from each positive sample and denotes it by $P_i^{(\star)}$. It then averages these descriptors and normalizes the result:

$$T_c = \frac{\sum_{i:y_i=c} P_i^{(\star)}}{\left\| \sum_{i:y_i=c} P_i^{(\star)} \right\|_2}.$$

The development set selects the block length from $M \in \{3, 4, 5, 6\}$. On the current dataset, the selected value is $M = 5$. The final templates are then re-estimated from same-class samples in the training and development sets. The whole procedure uses only framing, FFT, averaging, and normalized correlation, which keeps it close to the spectral analysis tools used in class.

At test time, the method applies the same descriptor to every candidate block and computes

$$\rho_{i,c}^{(r)} = \frac{\langle P_i^{(r)}, T_c \rangle}{\|P_i^{(r)}\|_2 \|T_c\|_2}, \qquad s_{i,c} = \max_r \rho_{i,c}^{(r)}.$$

The predicted class is $\arg\max_c s_{i,c}$. The reproducible algorithm is given in Appendix A.1.

Figure 1 shows the template construction used by our method. The left side gives representative training blocks near the class mean, and the right side gives the normalized average template for each consonant class.

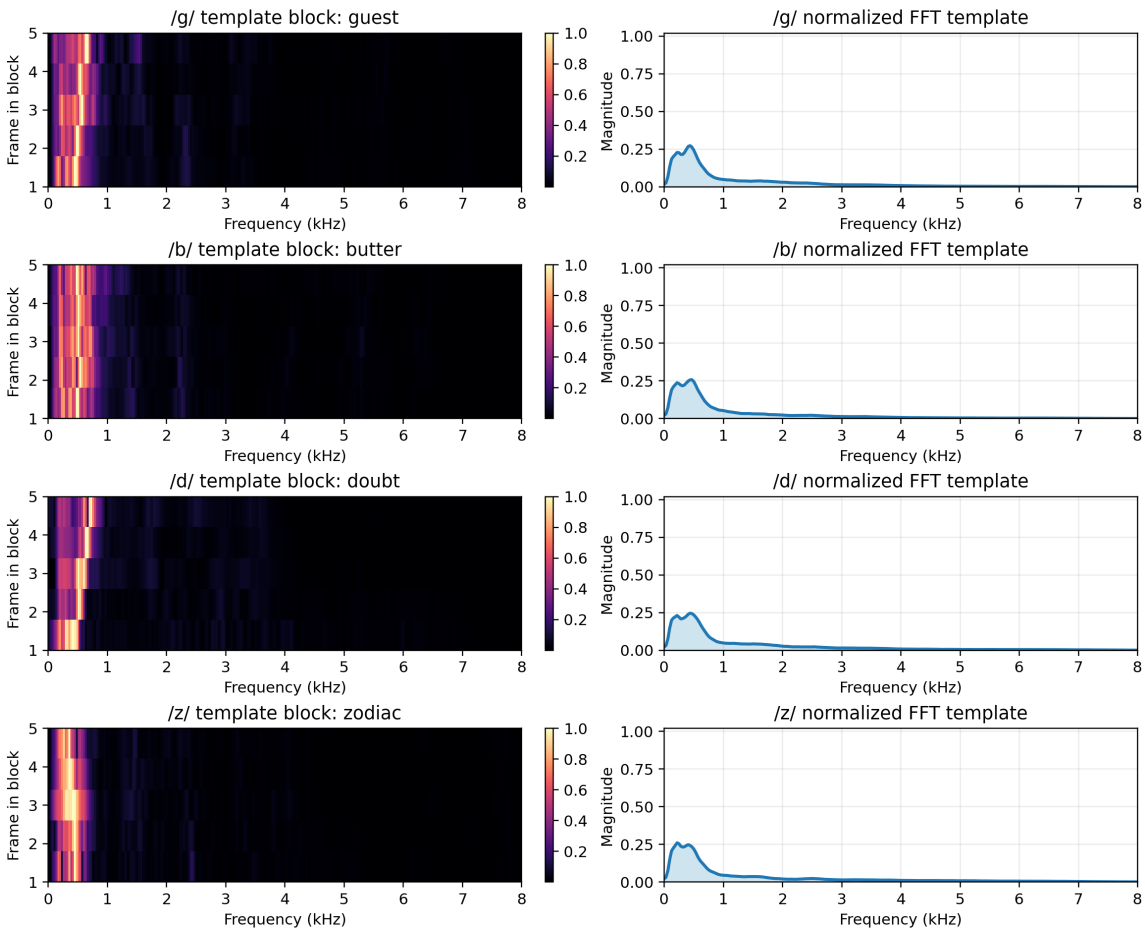

Figure 1: Template construction in our method. The maximum-energy block is extracted first, and the mean template is then computed from same-class samples in the training and development sets.

Figure 2 shows one correlation search. The upper panel is the waveform, the middle panel is the best-matching 5-frame block, and the lower panel shows the correlation score curves for the

four templates across candidate start positions. The procedure first searches for local evidence and then asks which template matches that evidence best.

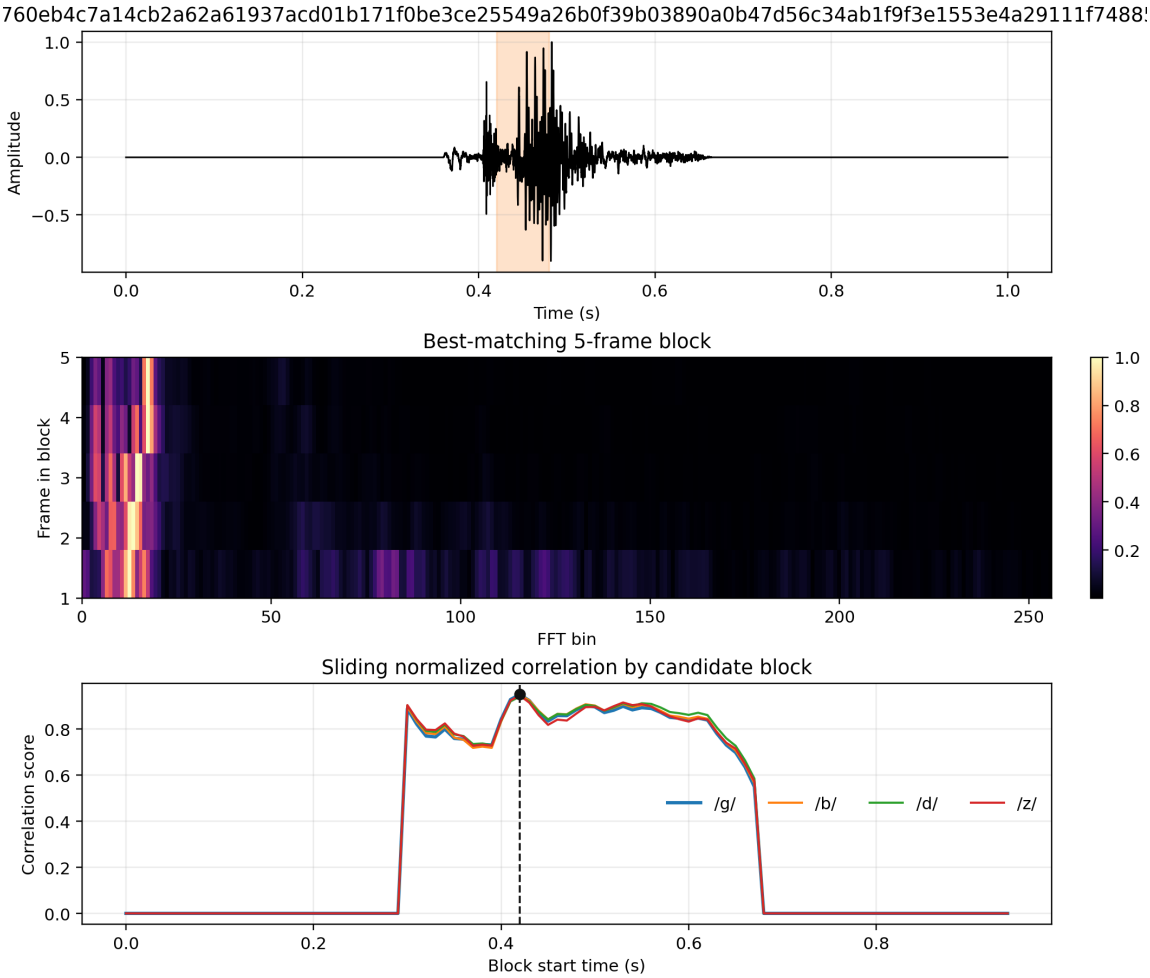

Figure 2: Correlation search in our method. The classifier scans candidate blocks and keeps the largest correlation score for each class.

4.2 AI reimplementation: filterbank-based local templates and autocorrelation

The AI reimplementation corresponds to `course_filterbank_corr`. It remains a template method, but it adds three changes: mel filterbanks for spectral compression, energy-based localization of a local onset window, and both positive and negative templates for each class. Its basic idea is that the most useful evidence lies near the word onset, a class should resemble its own positive samples and differ from the other three classes, and the energy-envelope autocorrelation can help separate sustained frication from short stop releases.

The method first uses 25 ms frames and 10 ms hops to compute the power spectrum and then applies 40 mel triangular filters to obtain a log-mel vector

$$L_i[m,q] = \log\left(\sum_k |X_{i,m}[k]|^2 B_q[k] + \epsilon\right), \qquad q = 1,\ldots,40.$$

Frame energy is defined as

$$e_i[m] = \log\left(\frac{1}{40}\sum_q \exp L_i[m,q] + \epsilon\right).$$

The code smooths $e_i[m]$ with a 5-frame moving average and uses the median of the first several frames as the baseline. The onset estimate o_i is the first position whose value exceeds the baseline by 0.22. The candidate window has length 16 frames, starts 2 frames before the onset estimate, and is also evaluated under three offsets $\Delta \in \{-2, 0, 2\}$:

$$W_i^{(\Delta)} = L_i[o_i - 2 + \Delta : o_i - 2 + \Delta + 15, :].$$

Each candidate window is divided into $S = 3$ segments. The method keeps both the raw segment means and the segment means after subtracting the baseline computed from the first 2 frames, and concatenates them into a local spectral vector

$$\phi_i^{(\Delta)} = \left[\mu_1(W), \mu_2(W), \mu_3(W), \mu_1(W - \bar{W}_{\text{pre}}), \mu_2(W - \bar{W}_{\text{pre}}), \mu_3(W - \bar{W}_{\text{pre}})\right].$$

Each μ_s is 40-dimensional, so $\phi_i^{(\Delta)} \in \mathbb{R}^{240}$. The method also computes a 24-lag normalized autocorrelation vector $a_i^{(\Delta)}$ from the energy envelope of the same window.

For each class c, the training set produces four templates: positive and negative local spectral templates P_c^+ and P_c^-, and positive and negative autocorrelation templates A_c^+ and A_c^-. These templates are the class-wise means of positive and negative training features. At test time, the score for each offset is

$$\begin{aligned} g_{i,c}^{(\Delta)} &= 0.65\left[\cos(\phi_i^{(\Delta)}, P_c^+) - \cos(\phi_i^{(\Delta)}, P_c^-)\right] \\ &\quad + 0.35\left[\cos(a_i^{(\Delta)}, A_c^+) - \cos(a_i^{(\Delta)}, A_c^-)\right], \\ s_{i,c} &= \max_{\Delta\in\{-2,0,2\}} g_{i,c}^{(\Delta)}. \end{aligned}$$

Compared with our method, this score adds negative-template contrast, autocorrelation, and a more explicit onset-localization step. In the scope of this report, it serves as the main AI reimplementation of the spectral description and detection stages. It still keeps an interpretable template structure, but the representation and the scoring rule are more complex. The reproducible algorithm is given in Appendix A.2.

Figure 3 shows the positive-negative contrasts that directly enter the classification rule. The first two columns show the template differences for raw log-mel segment means and onset-centered segment means, and the third column shows the positive-negative differences in energy-envelope autocorrelation.

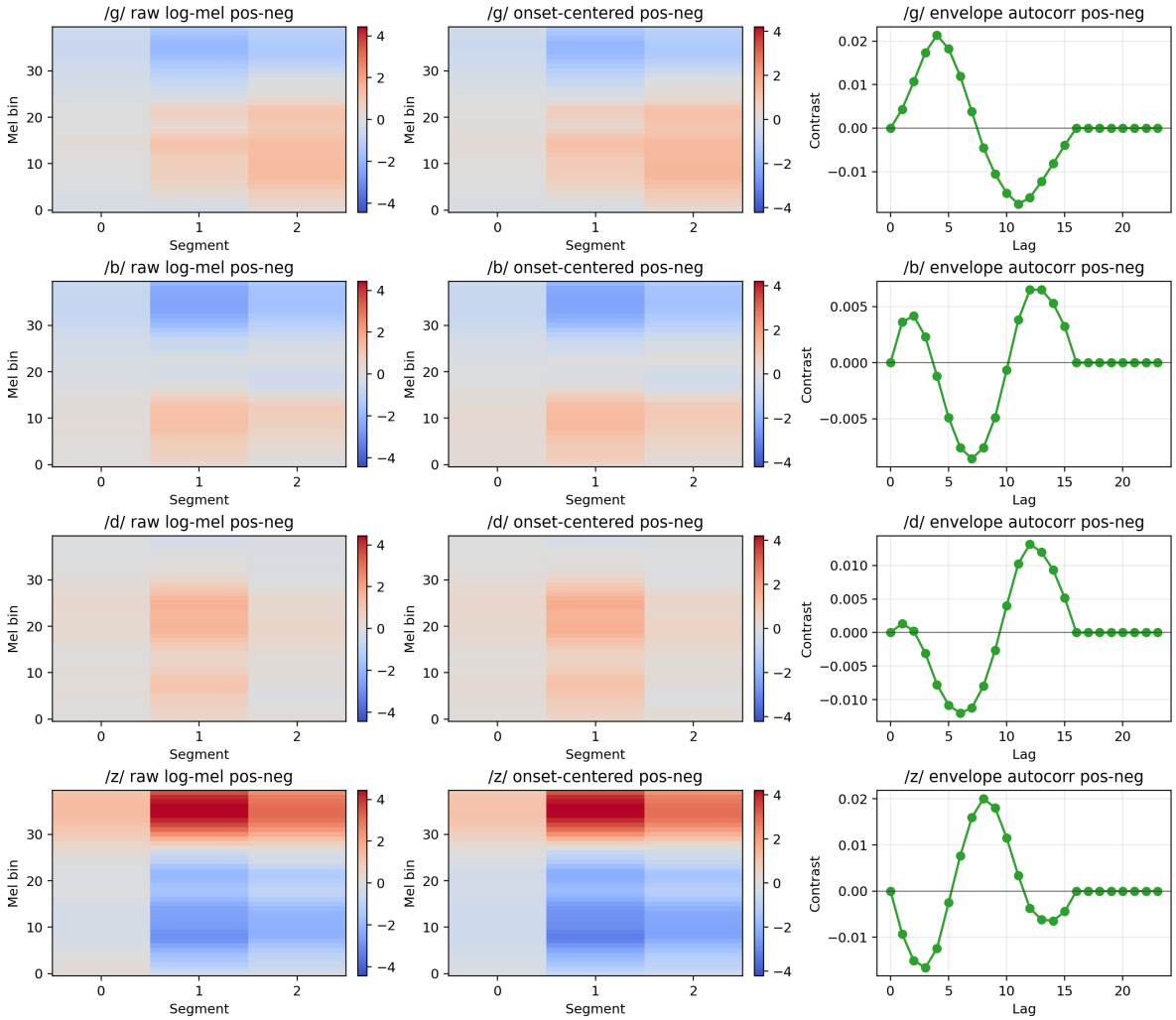

Figure 3: Positive-negative template differences in the AI reimplementation. Warmer regions indicate stronger similarity to positive samples, while cooler regions indicate stronger similarity to negative samples.

Figure 4 breaks one detection case into steps. The method first estimates the onset from log-mel energy, then extracts three candidate windows around that point, and finally computes positive-negative template scores with autocorrelation for each class and each offset.

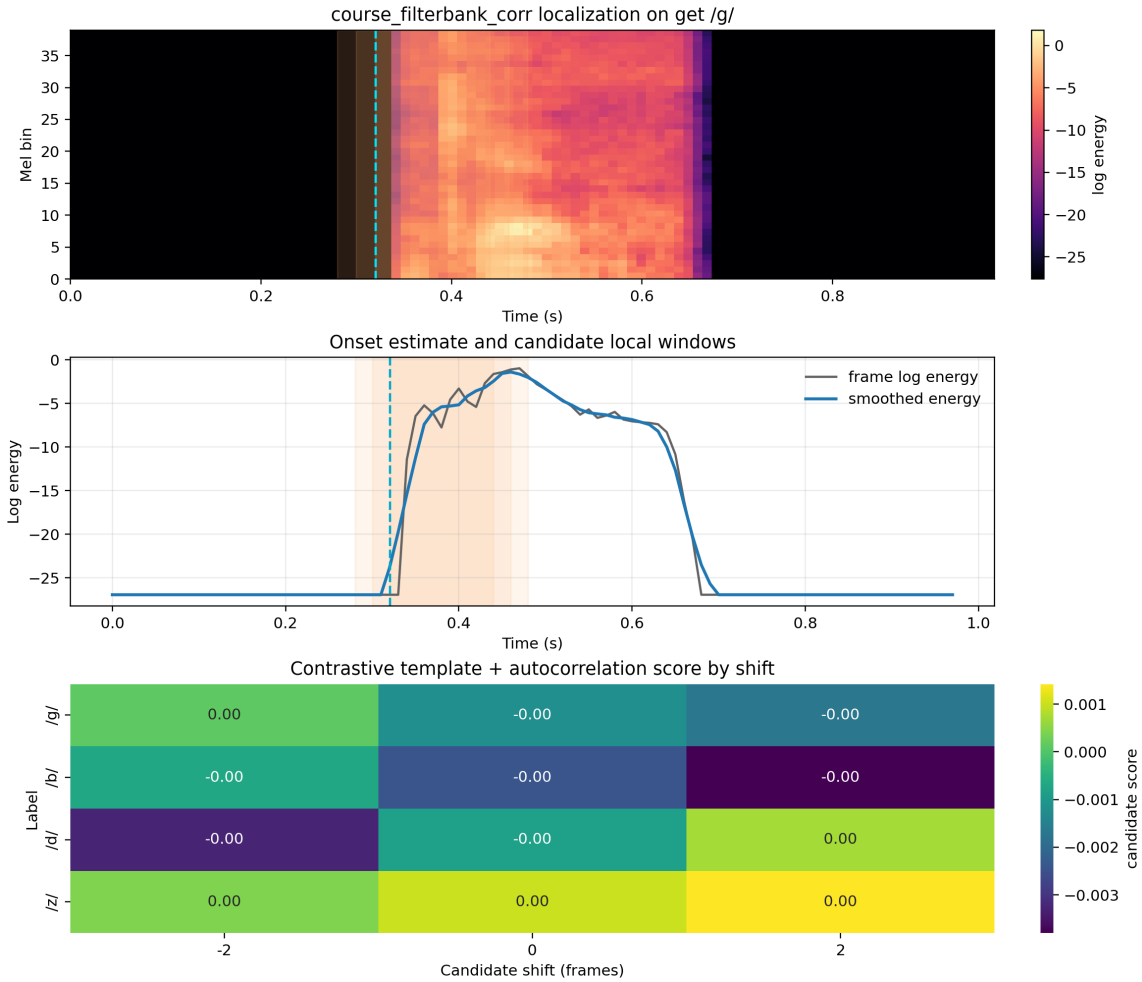

Figure 4: Local localization and offset scoring in the AI reimplementation.

4.3 MATLAB demonstration

To stay consistent with the spectral analysis tools used in class, we also implement a matching MATLAB demonstration. The core score remains

$$\texttt{score} = \frac{\texttt{mag}^\top \texttt{template}}{\|\texttt{mag}\|_2 \|\texttt{template}\|_2}.$$

The demonstration visualizes the short-time spectral templates and the correlation score matrix for four representative samples, which makes the relation between the templates and the speech signal easy to inspect. The formal evaluation, however, uses the shared data split, block-length selection on the development set, and template re-estimation from the training and development sets.

5 Case Analysis and Visualization

5.1 Spectral templates of four representative words

Figure 5 shows the short-time FFT templates of four representative words in the MATLAB demonstration. The figure is not a training summary. Its purpose is to show the actual shape of the local templates.

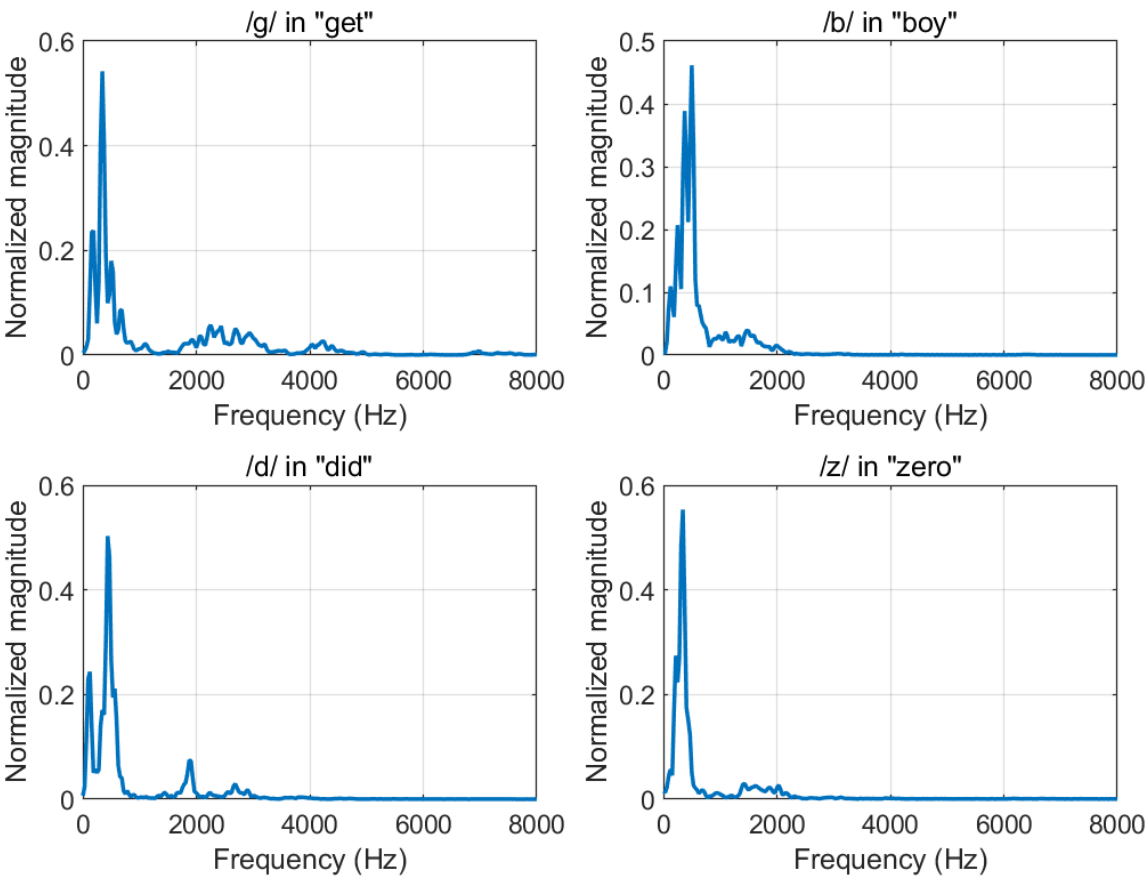

Figure 5: Short-time FFT magnitude templates of four representative words.

The figure shows a clear contrast. The high-frequency frication of /z/ lasts longer, the initial releases of /b/ and /d/ are shorter, and the short-time structure of /g/ is the easiest to blur when alignment shifts in time. This pattern is consistent with the class templates constructed by our method.

5.2 Correlation score matrix

Figure 6 gives the correlation score matrix produced by the same set of templates on the four representative words. The figure is not meant to prove that the model is strong. It shows that the basic operation of template-query correlation is actually working.

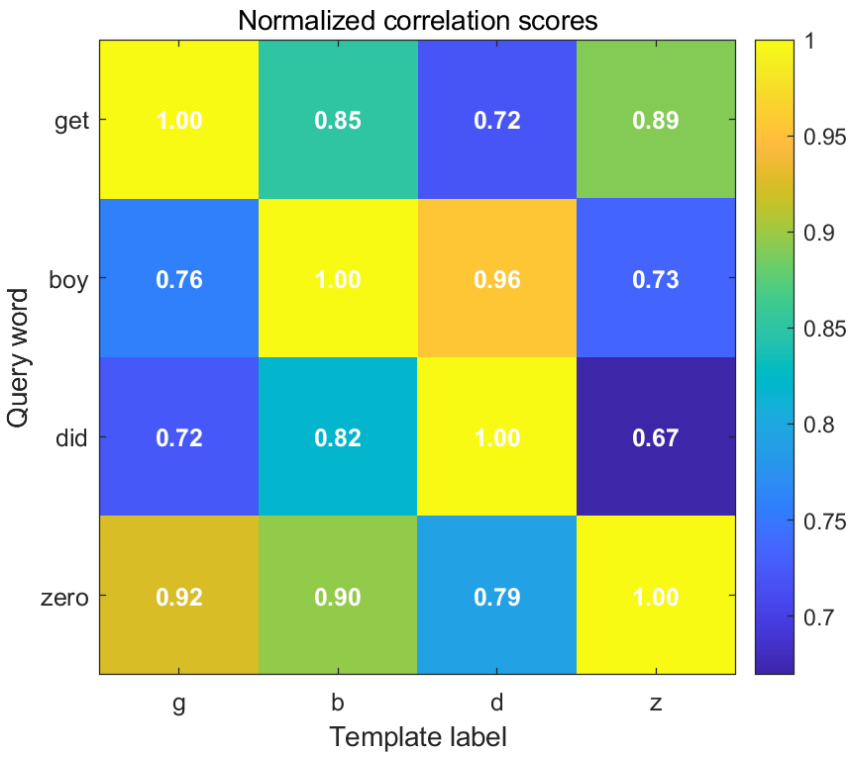

Figure 6: Correlation score matrix for four representative words.

This matrix is more direct than the waveform alone because it turns the question of which template looks most similar to a local speech segment into a numerical comparison. When the score gap is small, the alignment of the initial consonant is usually not stable enough. This is also the problem that the later AI reimplementation tries to address.

6 Experimental Results and Comparison

6.1 Our method

We first examine our short-time FFT template method. Since the task is single-label four-class classification, exact-match accuracy is the most direct metric, while label-wise accuracy is only a supplementary reference.

This method no longer uses only four representative case recordings as templates. It first averages the maximum-energy blocks of same-class samples in the training set and then re-estimates the final templates from the training and development sets after the block length is selected. The block length is chosen from 3 to 6 frames on the development set, and the selected value is 5. Because the data cover 300 words, cross-word and speaker variation are large, so FFT magnitude correlation alone cannot fully separate the four initial consonants. Even so, the method keeps a clear spectral interpretation.

Table 2: Main metrics of our short-time FFT template method.

Method	Label acc.	Exact match	Macro-F1	Micro-F1	Samples-F1
Our method	0.6327	0.2655	0.2439	0.2655	0.2655

Table 3: Per-class F1 of our short-time FFT template method.

Method	/g/	/b/	/d/	/z/
Our method	0.217	0.163	0.243	0.353

Figure 7 gives the score distributions for the four classes. Positive and negative samples show some separation, but the overlap is still large. This pattern indicates that short-time FFT templates alone are not stable enough for four-class recognition.

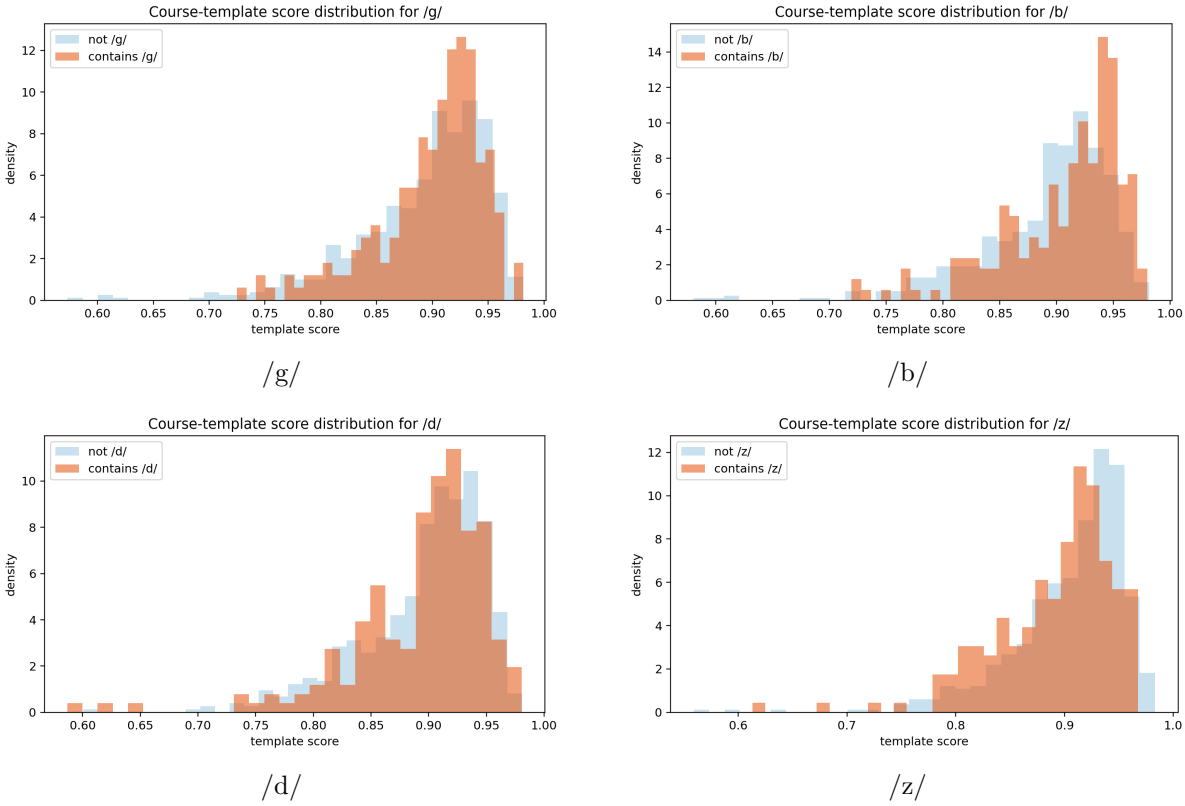

Figure 7: Score distributions of our method on the four classes.

6.2 AI reimplementation comparison

Beyond our method, the report also includes an AI reimplementation. It does not simply replace the classifier. It redoes the first two stages of the pipeline: it first uses log-mel features and energy change to obtain a more stable local spectrum near the word onset, and then uses positive-negative template differences and autocorrelation for detection.

Table 4: Overall comparison between our method and the AI reimplementation.

Method	Label acc.	Exact match	Macro-F1	Micro-F1	Samples-F1
Our method	0.6327	0.2655	0.2439	0.2655	0.2655
AI reimplementation	0.7513	0.5026	0.4974	0.5026	0.5026

Table 5: Per-class F1 of our method and the AI reimplementation.

Method	/g/	/b/	/d/	/z/
Our method	0.217	0.163	0.243	0.353
AI reimplementation	0.464	0.359	0.429	0.737

The gain of the AI reimplementation mainly comes from two changes. It narrows the search range more precisely to the region near the word onset, and it adds explicit positive-negative template contrast inside the local window. The largest gain appears on /z/, which suggests that sustained frication is more stable under local log-mel templates and autocorrelation. The overlap between /b/ and /d/ remains strong, which means that short stop releases and alignment error still weaken the template evidence.

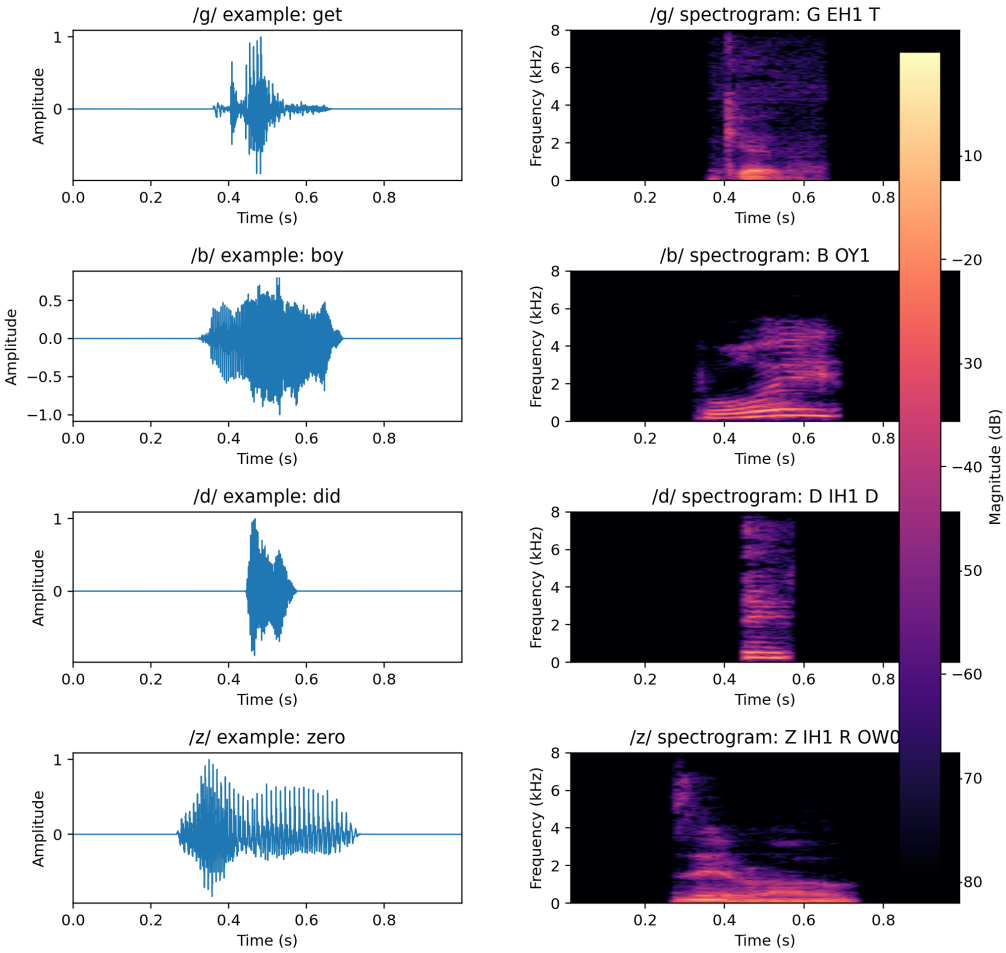

Figure 8: Four representative samples used in the AI reimplementation.

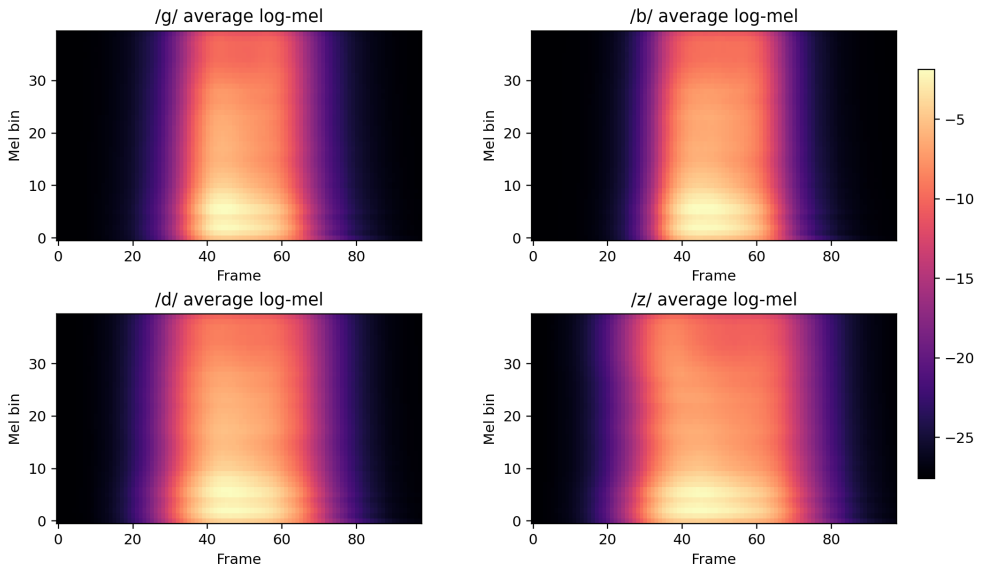

Figure 9: Average log-mel template view used for the AI reimplementation. The figure shows the mean spectrogram of each class. It is not identical to the positive-negative contrast templates, but it helps display the energy pattern near the word onset.

Figure 10 gives the test score distributions of the AI reimplementation for each class. Compared with our method, /z/ shows the clearest separation between positive and negative samples. The classes /g/, /b/, and /d/ still overlap strongly, which explains why the AI reimplementation improves the results but does not remove confusion among stop consonants.

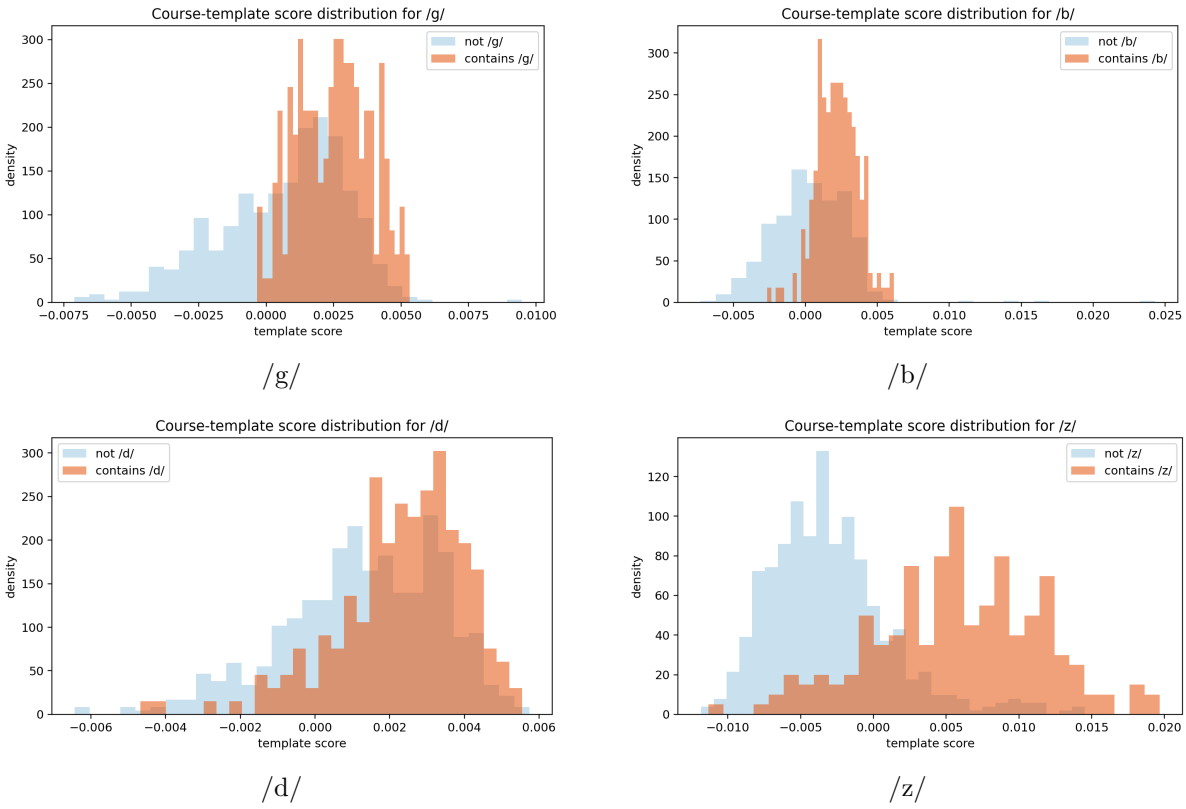

Figure 10: Score distributions of the AI reimplementation on the four classes.

7 Neural Baselines and Training Curves

After presenting our method and the AI reimplementation, this section adds the MLP and CNN as neural baselines. Both models use the same data split: the training set is used for parameter learning, the development set is used for threshold selection or early stopping, and the test set is used only for the final evaluation. The final prediction is obtained by arg max over the four output scores.

7.1 MLP: MFCC and band-energy statistics

The MLP does not take the full spectrogram as input. It uses handcrafted summary features. For each utterance, the pipeline computes log-mel features, extracts 13-dimensional MFCCs and their first-order differences, and concatenates five parts:

$$z_i = [\mathrm{mean}(\mathrm{MFCC}), \mathrm{std}(\mathrm{MFCC}), \mathrm{mean}(\Delta\mathrm{MFCC}), \mathrm{std}(\Delta\mathrm{MFCC}), \mathrm{bandEnergy}]\,.$$

The first four parts contribute $13 \times 4 = 52$ dimensions, and the band-energy term contributes 10 more, so $z_i \in \mathbb{R}^{62}$. All features are standardized by the mean and variance of the training set before learning begins.

The model is a two-hidden-layer feedforward network:

$$h_1 = \mathrm{ReLU}(W_1 z_i + b_1), \qquad h_2 = \mathrm{ReLU}(W_2 h_1 + b_2), \qquad p_i = \sigma(W_3 h_2 + b_3),$$

where the hidden widths are 128 and 64. The training loss is binary cross entropy over the four labels with an L_2 penalty:

$$\mathcal{L}_{\mathrm{MLP}} = -\sum_{i,c} [y_{i,c} \log p_{i,c} + (1 - y_{i,c}) \log(1 - p_{i,c})] + \lambda \|\theta\|_2^2.$$

The model assumes that whole-word MFCC statistics preserve enough average acoustic information. Since it does not localize the word onset, variation from the following vowel and from different speakers is mixed into the same representation. The algorithm is summarized in Appendix A.3.

7.2 CNN: whole-word log-mel spectrograms

The CNN directly uses the whole-word log-mel spectrogram. Let the input matrix be $L_i \in \mathbb{R}^{T \times 40}$. The matrix is standardized by the global mean and standard deviation of the training set and then treated as a single-channel image. The network structure is

$$\mathrm{Conv}(1, 24) \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{MaxPool},$$

$$\mathrm{Conv}(24, 48) \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{MaxPool},$$

$$\mathrm{Conv}(48, 96) \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{GlobalAvgPool} \to \mathrm{FC}(96) \to \mathrm{Dropout}(0.25) \to \mathrm{FC}(4).$$

The logits are denoted by $u_{i,c}$ and the probabilities by $p_{i,c} = \sigma(u_{i,c})$. The training loss is weighted binary cross entropy:

$$\mathcal{L}_{\mathrm{CNN}} = -\sum_{i,c} \alpha_c y_{i,c} \log p_{i,c} - (1 - y_{i,c}) \log(1 - p_{i,c}),$$

where α_c is computed from the class imbalance in the training set and clipped to $[1, 10]$. Optimization uses AdamW with learning rate 10^{-3}, weight decay 10^{-4}, and model selection by development-set macro-F1.

The CNN differs from the template methods because it does not define a fixed onset window in advance. Instead, it lets convolutional filters learn local patterns from the whole spectrogram. This assumption needs broader lexical coverage. Once the dataset expands to 300 words and 6796 utterances, the small CNN performs much better than it does in the earlier 60-word setting.

Table 6: Overall metrics of the neural baselines.

Method	Label acc.	Exact match	Macro-F1	Micro-F1	Samples-F1
MLP	0.7423	0.4845	0.4848	0.4845	0.4845
CNN	0.7796	0.5593	0.5557	0.5593	0.5593

The MLP is a stable statistical baseline, but it still falls below the AI reimplementation. The CNN benefits from the larger number of word types and training samples, and its exact-match accuracy reaches 0.559, which is higher than the 0.503 of the AI reimplementation. This result shows that once lexical coverage increases, an end-to-end spectrogram model begins to learn some local patterns near the word onset, although it remains weaker than template methods in interpretability.

Table 7: CNN epoch ablation.

Maximum epoch	Completed epoch	Best dev epoch	Dev macro-F1	Test exact match
5	5	5	0.529	0.372
10	10	8	0.575	0.499
15	15	11	0.598	0.460
20	20	18	0.598	0.523
25	25	25	0.608	0.559
30	30	26	0.614	0.541
40	34	26	0.614	0.541

In terms of development-set macro-F1, the 30-epoch and 40-epoch settings both reach their best point near epoch 26, but the 40-epoch setting stops early at epoch 34 and brings no extra gain. In terms of test-set exact-match accuracy, the 25-epoch setting is the best. For this reason, the report uses 25 epochs as the final CNN configuration and treats the range from 25 to 30 epochs as a suitable training interval for the current data scale.

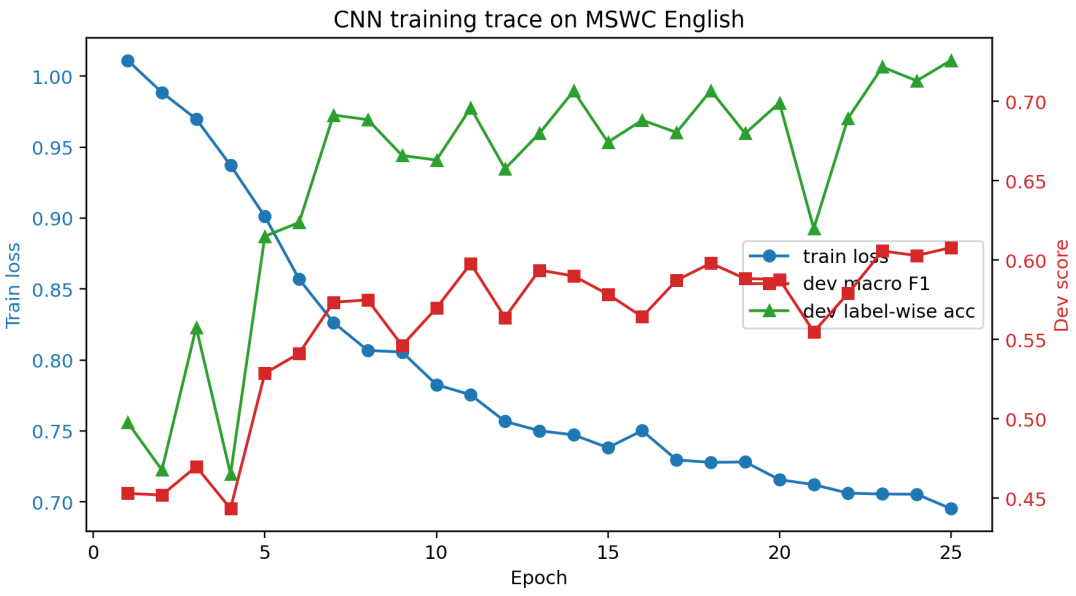

Figure 11: CNN training curves.

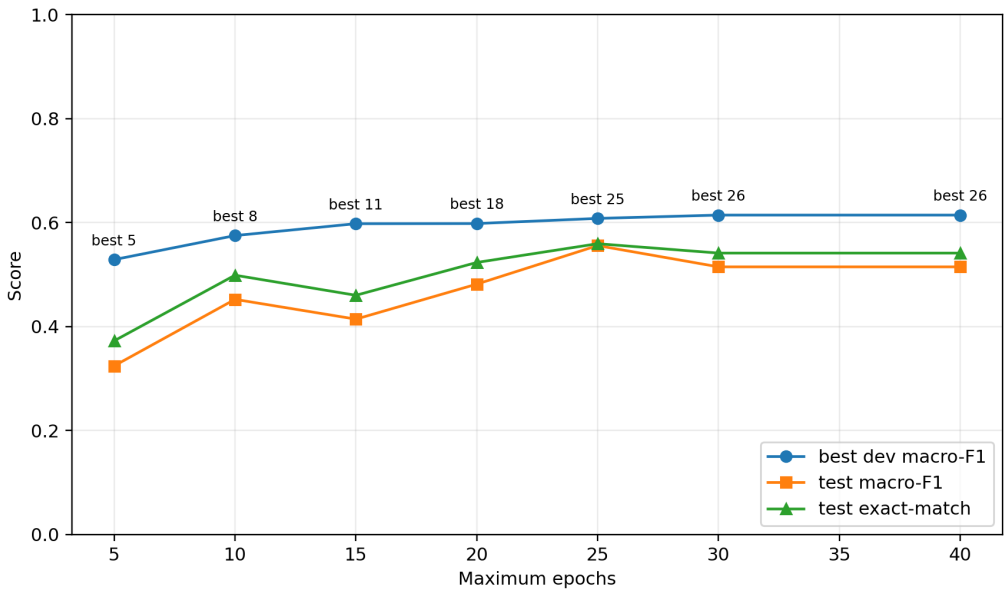

Figure 12: CNN maximum-epoch ablation. Test exact-match accuracy peaks at 25 epochs, while development-set macro-F1 is slightly higher at 30 or 40 epochs and does not improve the test result.

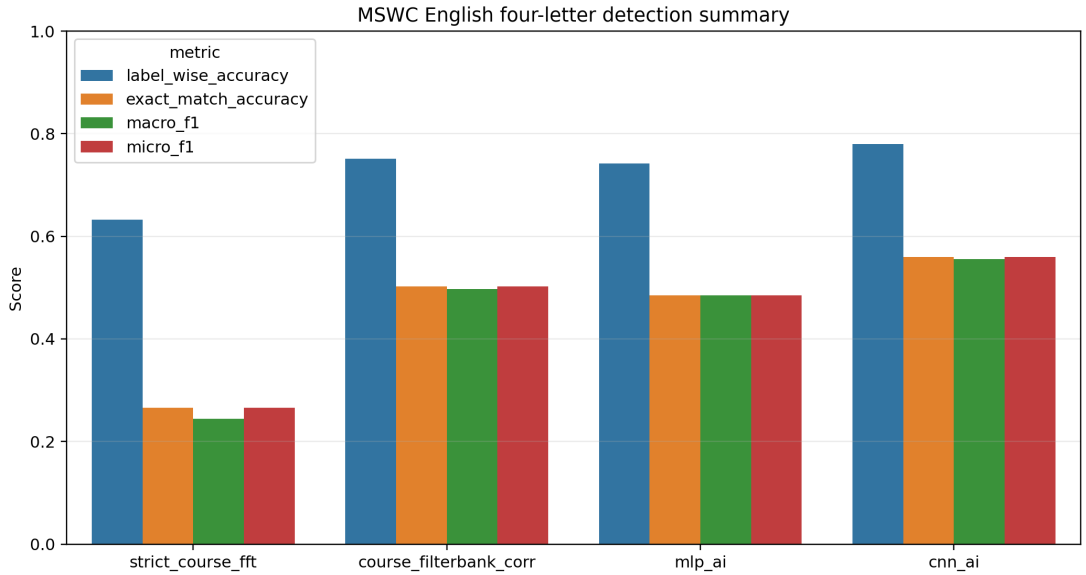

Figure 13: Overall metric summary of all methods.

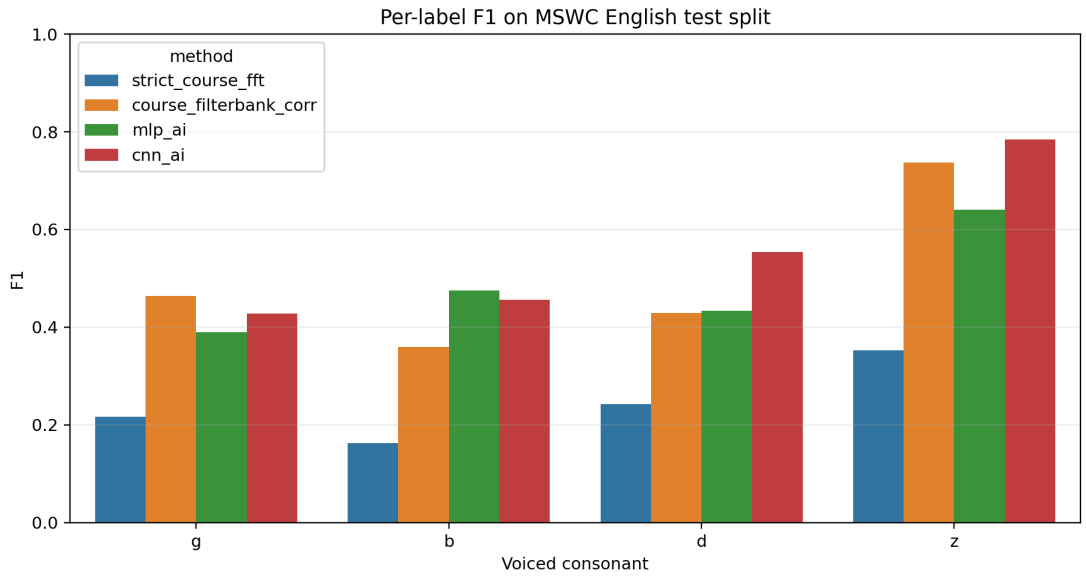

Figure 14: Per-class F1 comparison of all methods.

Overall, the neural models improve the final scores once the dataset becomes larger, but this gain mainly appears in discriminative power rather than in acoustic interpretability.

8 Discussion and Limitations

The four methods form a fairly clear order. Our method reaches 0.265 exact-match accuracy and 0.244 macro-F1, which is close to random four-class performance. This result shows that framing, FFT, local blocks, and normalized correlation can form a complete detection pipeline, but they are not enough for stable four-class discrimination. The intermediate examples explain why: for samples such as `get` and `boy`, the four FFT correlation scores are often very close, which

means that the maximum-energy block and the one-sided magnitude-spectrum template are not sufficient to separate /g/, /b/, and /d/ reliably.

The AI reimplementation raises exact-match accuracy to 0.503 and macro-F1 to 0.497. This improvement does not come from a black-box classifier. It comes from a more suitable intermediate representation. Log-mel compression smooths spectral shape, onset-centered local windows reduce interference from the vowel in the rest of the word, positive-negative template contrast suppresses energy shared across classes, and autocorrelation adds time-structure information that helps distinguish sustained frication from short stop releases. The class-wise results follow the same explanation. The F1 of /z/ reaches 0.737 because its frication lasts longer and stays more stable. The F1 of /b/ is only 0.359, which shows that a weak release and the following vowel still make it easy to confuse /b/ with /g/ and /d/.

The MLP and CNN are added to examine whether learned models can go beyond the AI reimplementation. The MLP reaches 0.485 exact-match accuracy, which is lower than the AI reimplementation, so whole-word MFCC statistics remain too coarse and average in variation from the word-final vowel and the speaker. The CNN reaches 0.559 exact-match accuracy, which is the best result in the current comparison. It is much stronger on /d/ and /z/, which suggests that after the dataset expands to 300 words and 6796 utterances, the convolutional model does learn some local patterns from the full log-mel spectrogram. Even so, the epoch ablation also shows that the highest development-set macro-F1 at 30 or 40 epochs does not produce the best test accuracy. The best test result still appears at 25 epochs, so the gain of the CNN remains partly dependent on training variance and the data split.

For this reason, ranking the methods by the highest score alone does not tell the whole story. Our method has the lowest score but the clearest interpretation. The AI reimplementation is not the top-scoring method, but it gives the clearest explanation of why a better spectral representation and a better detection structure help. The CNN has the highest score, but its interpretability is the weakest. The comparison also shows that label-wise accuracy is inflated by true negatives under one-hot four-class decoding, so exact-match accuracy, macro-F1, and per-class F1 deserve more attention.

The current study still has several limits. The input remains whole-word audio rather than manually segmented phoneme clips, so our method must rely on sliding-window search. The release cues of /g/, /b/, and /d/ are short, and even small alignment error can bury the template evidence under the following vowel. The current AI reimplementation is still a template method, so it does not use phone-level alignment and does not explicitly model speaker or channel variation. The current CNN is only a light comparison model. It does not include data augmentation, pretrained speech models, or forced alignment, so it should not be treated as an upper bound for deep speech models.

9 Conclusion

This report studies spectral description and detection for the initial voiced consonants /g/, /b/, /d/, and /z/. After filtering the data by actual pronunciation, the dataset contains 75 words per class, for a total of 300 words and 6796 utterances. This definition gives a clearer label boundary than a filter based on spelling alone.

On this dataset, our method forms a complete detection pipeline by using only short-time FFT magnitude spectra and normalized correlation. The best block length selected on the development set is 5 frames, and the resulting test exact-match accuracy is 0.265. The AI reimplementation raises exact-match accuracy to 0.503 by adding a local log-mel onset window, positive-negative template contrast, and autocorrelation. The MLP reaches 0.485 exact-match

accuracy, and the CNN reaches 0.559. In overall score, the CNN performs best, but the template methods provide a clearer view of the acoustic evidence. Across all methods, /z/ is the easiest class, while the short stop releases of /g/, /b/, and /d/ remain the main source of error because they depend more strongly on alignment.

A Algorithm Structure and Pseudocode

This appendix summarizes the four implementations used in the report in a reproducible form. All methods use the same input setting: 16 kHz, 1 s, mono WAV recordings, the label set $C = \{g, b, d, z\}$, and one shared train/development/test split in which the three subsets are used only for training, model selection, and final testing, respectively.

A.1 `strict_course_fft`

The method assumes that each initial voiced consonant can be represented by a short-time FFT magnitude-spectrum prototype. The maximum-energy block approximates the main local evidence of the initial consonant, and normalized correlation reduces the effect of recording level. In practice, each training waveform is framed with 20 ms windows, 10 ms hops, and a 512-point FFT. The method computes a normalized magnitude-spectrum descriptor $P_i^{(r)}$ for every candidate M-frame block, selects the maximum-energy block $P_i^{(\star)}$ from each positive training sample, and averages same-class descriptors to obtain the class template T_c. The block length is chosen from $M \in \{3, 4, 5, 6\}$ on the development set by exact-match accuracy, after which the templates are re-estimated from the training and development sets. At test time, the method slides over all candidate blocks, computes $s_{i,c} = \max_r \cos(P_i^{(r)}, T_c)$, and predicts $\hat{c}_i = \arg\max_c s_{i,c}$.

A.2 `course_filterbank_corr`

This method assumes that a local log-mel window near the onset is more reliable than a whole-word average, that positive-negative template contrast suppresses energy shared across classes, and that energy-envelope autocorrelation helps distinguish sustained frication from short stop releases. The waveform is first framed with 25 ms windows and 10 ms hops to produce a 40-bin log-mel spectrogram. The onset o_i is estimated from the smoothed frame-energy curve. Three candidate 16-frame windows are then extracted with offsets $\Delta \in \{-2, 0, 2\}$. Each local window is divided into three segments, and the method concatenates raw segment means with baseline-subtracted segment means to form $\phi_i^{(\Delta)}$. It also computes a 24-lag normalized autocorrelation vector $a_i^{(\Delta)}$ from the same local window. For each class, the training set is used to average positive and negative samples separately, which yields P_c^+, P_c^-, A_c^+, and A_c^-. The score at each offset is computed as $0.65[\cos(\phi, P_c^+) - \cos(\phi, P_c^-)] + 0.35[\cos(a, A_c^+) - \cos(a, A_c^-)]$. The maximum over the three offsets becomes $s_{i,c}$, and the final prediction is $\arg\max_c s_{i,c}$.

A.3 `mlp_ai`

This model assumes that the mean and standard deviation of MFCCs, the statistics of delta MFCCs, and coarse band-energy features already summarize the main acoustic differences in the whole word. The network structure is

$$62 \to 128 \to 64 \to 4.$$

The hidden layers use ReLU, and the output layer uses sigmoid probabilities. Training uses Adam, early stopping, and L_2 regularization. The procedure first extracts MFCC statistics, delta-MFCC statistics, and 10-dimensional band energy from each recording. It then standardizes the features with the mean and variance of the training set, trains the two-layer MLP with binary cross entropy over the four labels, estimates per-class thresholds on the development set while preserving exclusive decoding, and finally outputs four probabilities on the test set and selects the largest one.

A.4 `cnn_ai`

This model assumes that the log-mel spectrogram can be treated as a time-frequency image and that convolutional layers can learn useful local acoustic patterns from the whole word without a hand-designed onset window. The structure is

$$\begin{aligned}
&\mathrm{Conv}_{24} \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{Pool} \\
&\to \mathrm{Conv}_{48} \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{Pool} \\
&\to \mathrm{Conv}_{96} \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{GlobalAvgPool} \\
&\to \mathrm{FC}_{96} \to \mathrm{Dropout} \to \mathrm{FC}_{4}.
\end{aligned}$$

The procedure first extracts whole-word log-mel spectrograms and standardizes them with the global mean and variance of the training set. The spectrograms are then passed as single-channel inputs into the three-stage convolutional network. Training uses positive-weighted `BCEWithLogitsLoss` and AdamW. After each epoch, the model is evaluated on the development set by macro-F1, and the checkpoint with the best development performance is kept. At test time, the logits are converted into sigmoid probabilities, and the class with the largest probability is selected.

References

[1] S. Davis and P. Mermelstein, "Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1980. https://doi.org/10.1109/TASSP.1980.1163420.

[2] L. R. Rabiner, "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition," *Proceedings of the IEEE*, 1989. https://doi.org/10.1109/5.18626.

[3] N. J. Wilkinson and M. J. Russell, "Improved Phone Recognition on TIMIT Using Formant Frequency Data and Confidence Measures," in *Proc. ICSLP 2002*. https://www.isca-archive.org/icslp_2002/wilkinson02_icslp.html.

[4] M. Mazumder *et al.*, "Multilingual Spoken Words Corpus," in *Proc. NeurIPS Datasets and Benchmarks*, 2021. https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/fe131d7f5a6b38b23cc967316c13dae2-Abstract-round2.html.

[5] G. Chen, C. Parada, and G. Heigold, "Small-Footprint Keyword Spotting using Deep Neural Networks," in *Proc. ICASSP 2014*. https://research.google/pubs/small-footprint-keyword-spotting-using-deep-neural-networks/.

[6] T. N. Sainath and C. Parada, "Convolutional Neural Networks for Small-Footprint Keyword Spotting," in *Proc. Interspeech 2015*. https://research.google/pubs/convolutional-neural-networks-for-small-footprint-keyword-spotting/.

[7] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv:1804.03209, 2018. https://doi.org/10.48550/arXiv.1804.03209.

[8] M. Berg *et al.*, "Keyword Transformer: A Self-Attention Model for Keyword Spotting," arXiv:2104.00769, 2021.

[9] A. Baevski *et al.*, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," arXiv:2006.11477, 2020. https://doi.org/10.48550/arXiv.2006.11477.

[10] W.-N. Hsu *et al.*, "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units," arXiv:2106.07447, 2021.

[11] S. Chen *et al.*, "WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing," *IEEE Journal of Selected Topics in Signal Processing*, 2022. https://arxiv.org/abs/2110.13900.

[12] C. Gao *et al.*, "Self-Supervised Speech Representation Learning for Keyword-Spotting with Light-Weight Transformers," arXiv:2303.04255, 2023. https://arxiv.org/abs/2303.04255.

[13] D. Kim and J. Hwang, "Fully End-to-end Streaming Open-vocabulary Keyword Spotting with W-CTC Forced Alignment," in *Proc. Interspeech 2025*. https://www.isca-archive.org/interspeech_2025/kim25d_interspeech.html.